Information Sheet 1: Introduction to Environmental Modelling

EART22001 / EART60071

1 Why start with ordinary differential equations?

We will begin the course with models that solve *ordinary differential equations* (ODEs). You will have met ODEs in the Natural Scientists Toolkit. An ODE describes how one or more quantities change, often with time.

A simple example is

$$\frac{dy}{dt} = -\lambda y,$$

with the initial condition $y(0) = N_0$. Separating variables and integrating gives

$$y = N_0 \exp(-\lambda t),$$

which you may recognise as radioactive decay.

The important modelling idea is not the particular solution. We start from a rule for the *rate of change*, provide an initial state, and calculate how the system evolves. Many environmental problems can be approached in this way.

Some differential equations can be solved exactly; many cannot. Numerical methods approximate their solutions on a computer. A simple example is the forward Euler method. More accurate algorithms are available, and in this module we will concentrate on using and interpreting models rather than deriving every numerical algorithm. For further detail, see Hoffman (1992).

2 What counts as an environmental model?

There is no single kind of environmental model. During this course you will meet several useful families.

Type	**What it means in this course**
ODE models	Models in which an initial state evolves with time. Examples include orbital motion and the evolution of atmospheric particles, cloud droplets or chemical species.
Empirical models	Models that are guided by physical ideas but contain parameters obtained from observations. The Gaussian plume model is an example used in air-quality work.
Reduced-dimensional models	Deliberately simplified models that represent only the dimensions needed for a question. A single-column atmospheric model, for example, represents vertical structure without explicitly modelling the full three-dimensional atmosphere.
Fluid-dynamical models	Models based on equations governing the motion of fluids. These ideas underpin weather, ocean and many other environmental models.

A model is therefore not automatically "good" because it is complicated. A useful model is one whose assumptions and level of complexity are appropriate to the question being asked.

3 How we will work

For most practicals the numerical model will run on a **Linux server**. Your Windows PC or Mac is the *local* computer. The server is the *remote* computer.

The basic pattern is:

1. connect from your computer to the server using **SSH**;
2. obtain or update the practical files, often using **Git**;
3. edit input files and run the model on the server;
4. inspect the output;
5. transfer results back to your own computer when needed;
6. analyse, plot and interpret the results.

Do not worry if the command line is unfamiliar. The first practical is designed to teach the small set of commands you need. Keep the separate *Tools of the Trade* guide nearby: it is a reference document, not something you need to memorise before class.

---

4 A few terms you will hear

Term	Meaning
Terminal	A text-based window in which you type commands. On macOS use Terminal; on Windows use PowerShell, Command Prompt or Windows Terminal.
Linux	The operating system used by the course modelling server.
SSH	A secure way to log in to the remote server and run commands there.
SFTP / SCP	Secure ways to move files between the server and your own computer.
Computer code	Instructions written in a programming language. You will use both interpreted and compiled programs during the course.
Git	A version-control system used to keep track of changes to files and code.
GitHub	An online service that hosts Git repositories. We will often obtain practical code from GitHub.
Docker	An optional way to run the course computing environment locally. It is useful for some off-campus work but is not required for the normal on-campus workflow.

5 Before the first practical

Please do the following before class:

- ☐ Read this information sheet.
- ☐ Locate the separate *Tools of the Trade* guide on Canvas.
- ☐ Locate your **course server username and password** on Canvas. These are not your central University username and password.
- ☐ Make sure you know how to open PowerShell/Windows Terminal on Windows, or Terminal on macOS.

You do **not** need to install Docker before the first on-campus practical.

Reference: Hoffman, J. D. (1992), *Numerical Methods for Engineers and Scientists*, McGraw-Hill.

Tools of the Trade

EART22001 / EART60071

This section is a practical guide to the computing environment used in the modelling practicals for EART22001 and EART60071. You should be able to work through it from the beginning on either Windows or macOS, even if you have not used a command line before.

For most practicals, the numerical model will run on a **Linux server**. Your laptop or desktop is the computer you sit in front of; it is used to connect to the server, transfer files and inspect results. Once you connect with SSH, the commands you type are being run on the remote Linux machine.

1 What you need before you start

Have the following ready:

- a Windows or macOS computer with an internet connection;
- the on-campus server address, `130.88.55.57`;
- the **course server username and password** shown on Canvas;
- a terminal application: PowerShell, Command Prompt or Windows Terminal on Windows, or Terminal on macOS.

The server login is not your central University login. Use the course-specific username and password supplied on Canvas. Do not put passwords into scripts, screenshots, Git repositories or discussion posts.

For on-campus practicals, the server address is `130.88.55.57`. Canvas remains the authoritative source for the username/password and for any announcement that the address has changed.

In the examples below, `YOUR_USERNAME` means the course username supplied on Canvas and `SERVER_ADDRESS` means `130.88.55.57` when you are working on campus. When a terminal example shows a prompt, the prompt is only a visual cue: type the command that follows it, not the prompt itself.

Commands, filenames, paths and literal messages are shown in a shaded `monospace style`. Larger shaded boxes contain complete commands or terminal sessions: type only the command text, not the surrounding box.

2 The basic workflow

Most practicals follow the same pattern:

1. open a terminal on your own computer;
2. connect to the course server using **SSH**;
3. move to the directory for the practical and obtain the code or data;
4. edit an input or configuration file if required;
5. run the model on the server;
6. inspect the output;
7. transfer any results you want to keep back to your own computer;
8. analyse or plot the results, usually with Python.

A useful distinction throughout this guide is:

> **Local computer** = your Windows PC or Mac.
> **Remote computer** = the Linux modelling server.
>
> SSH gives you a terminal *on the remote computer*. Commands such as `scp` that copy files to or from your laptop are normally started from a *local* terminal.

3 Windows: connect with PowerShell, Command Prompt or Windows Terminal

Windows Terminal is an application that can host several command-line shells. For this course, **PowerShell** is a good default, but Command Prompt works too.

1. **Open a terminal.** Press the Windows key, type **PowerShell** or **Terminal**, and open the application.
2. **Check that SSH is available.** Type:

```
ssh -V
```

If you see an OpenSSH version, you are ready. If Windows reports that `ssh` is not recognised, the OpenSSH Client may need to be enabled in Windows Optional Features. On a University-managed computer, do not change system settings without permission; ask a graduate teaching assistant (GTA) if the client is not available. See the Microsoft OpenSSH documentation if you need Windows-specific installation help.

3. **Connect to the course server.**

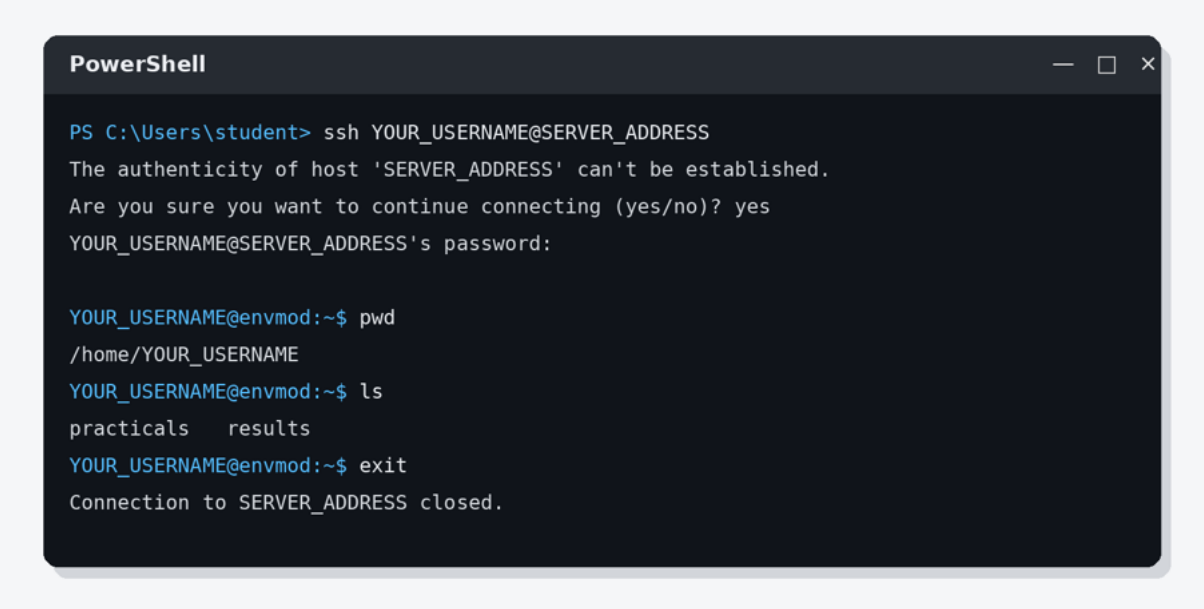
```
ssh YOUR_USERNAME@SERVER_ADDRESS
```

For example, on campus this is:

```
ssh YOUR_USERNAME@130.88.55.57
```

The first time you connect to a server, SSH may ask whether you trust its host key. Check that the server address exactly matches Canvas before answering `yes`. You will then be asked for the course password.

Nothing appears while you type an SSH password. There are no dots or asterisks. This is deliberate; type the password and press Enter.



Illustrative PowerShell session. Your prompt, username, server name and first-connection message will differ.

Once the remote prompt appears, you are logged in to Linux. Try:

```
pwd
ls
```

To finish the remote session cleanly, type `exit`.

4 macOS: connect with Terminal

macOS includes both Terminal and the SSH command-line tools.

1. **Open Terminal.** Press Command–Space to open Spotlight, type **Terminal**, and press Return. You can also find it in `/Applications/Utilities`.
2. **Check SSH.**

```
ssh -V
```

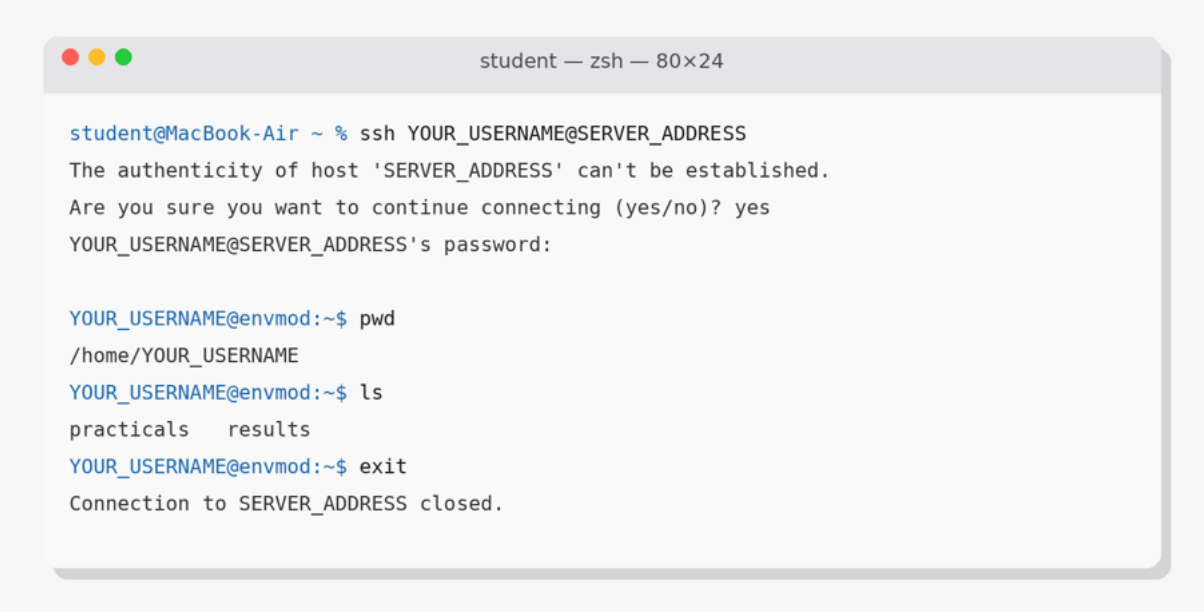
3. Connect.

```
ssh YOUR_USERNAME@SERVER_ADDRESS
```

On campus, use:

```
ssh YOUR_USERNAME@130.88.55.57
```

As on Windows, check the server name carefully if you are shown a first-connection host-key question, then enter the course password. The password will not be displayed while you type it. Apple provides a concise Terminal User Guide if you want more detail.



Illustrative macOS Terminal session. The default macOS shell is commonly zsh, while the remote server prompt shown after login is Linux.

Type `exit` when you want to disconnect from the server.

5 How to tell whether you are local or remote

This is one of the most important habits to develop. Look at the prompt before running a command.

Typical prompts look like this:

Where you are	Example prompt
Windows (local)	`PS C:\Users\student>`
macOS (local)	`student@MacBook-Air ~ %`
Linux server (remote)	`YOUR_USERNAME@envmod:~$`

If you are unsure where you are, type `hostname` and `pwd`. The former tells you which computer is running the shell; the latter tells you which directory you are in.

6 Essential Linux commands on the server

You do not need a large Linux vocabulary. The commands below cover most routine work in either module.

Important: words in CAPITALS below are placeholders, not words to type literally. For example, `cd DIR` means "type `cd` followed by the name of the directory you actually want", such as `cd practical1`. Likewise, `mv OLDNAME NEWNAME` could become `mv output.dat output_old.dat`.

Command pattern	What it does
`pwd`	print the full path of the current directory
`ls`	list files and directories
`ls -lh`	list files with readable sizes and more detail
`cd DIR`	enter a directory, for example `cd practical1`
`cd ..`	move up one directory level
`cd ~`	return to your home directory
`mkdir NAME`	create a directory, for example `mkdir results`
`nano -l FILE`	edit a text file in nano and display line numbers
`less FILE`	read a text file one screen at a time; press `q` to quit
`cp SOURCE TARGET`	copy a file, for example `cp input.txt input_backup.txt`
`mv OLDNAME NEWNAME`	move or rename a file, for example `mv run1.dat run1_old.dat`
`rm -i FILE`	remove a file, asking you to confirm before deletion
`hostname`	show the name of the computer you are logged into
`exit`	leave the remote SSH session

> **Be careful with `rm`.** On the Linux server, deleted files do not normally go to a recycle bin. For beginners, `rm -i FILE` is preferable because it asks for confirmation. Plain `rm FILE` removes the file immediately. Before deleting anything important, use `pwd` and `ls` to make sure you are in the directory you think you are in and that the filename is correct.

Three keyboard habits save a lot of time:

- **Tab completion:** type the first few letters of a file or directory and press Tab. The shell will complete it when possible.
- **Up arrow:** recalls previous commands, which is safer than retyping a long command.
- **Ctrl+C:** asks a running command to stop. Use this when you started the wrong command or a program is waiting indefinitely.

Linux file names are case-sensitive: `Results.csv` and `results.csv` are different files. Avoid spaces in working-directory and file names unless you are comfortable quoting them.

7 Editing a file with nano

For small configuration files, `nano` is sufficient and runs entirely in the SSH terminal. I recommend using the `-l` option so that line numbers are displayed down the left-hand side:

```
nano -l input.txt
```

This is particularly useful when a practical tells you to change a particular line or when you are comparing your file with a worked example. Edit the text normally. At the bottom of nano, the symbol ^ means the Ctrl key. The most useful shortcuts are:

- **Ctrl+O**, then Enter: save ("write out") the file;
- **Ctrl+X**: exit nano;
- **Ctrl+W**: search within the file.

8 Moving files between your computer and the server

There are two straightforward command-line approaches: `scp` for quick copies and `sftp` for an interactive file-transfer session. Both use SSH and the same course credentials.

Quick copies with scp

Run these commands in a **local** PowerShell/Command Prompt/Terminal window, not inside the SSH session.

Copy a local file to your server home directory:

```
scp myfile.py YOUR_USERNAME@SERVER_ADDRESS:~/
```

Copy a file from the server into the current local directory:

```
scp YOUR_USERNAME@SERVER_ADDRESS:~/results.csv .
```

Copy a whole directory from the server:

```
scp -r YOUR_USERNAME@SERVER_ADDRESS:~/results .
```

Here `~/` means your home directory on the server and the final dot means "the current directory on my local computer". If you are unsure where that is, run `pwd` in your *local* terminal before the `scp` command. After the transfer, `explorer .` on Windows or `open .` on macOS opens that folder graphically.

Browsing with sftp

Start SFTP from a local terminal:

```
sftp YOUR_USERNAME@SERVER_ADDRESS
```

Once connected, remember that commands without an `l` refer to the **remote** server; commands beginning with `l` refer to the **local** computer.

SFTP command	Meaning
`pwd` / `ls`	show the remote directory / list remote files
`cd DIR`	change remote directory
`lpwd` / `lls`	show the local directory / list local files
`lcd DIR`	change local directory
`get FILE`	download a remote file into the current local directory
`put FILE`	upload a local file from the current local directory
`exit`	close SFTP

A useful way to think about SFTP is that it keeps track of **two directories at the same time**:

- `pwd` tells you the remote directory on the Linux server;
- `lpwd` tells you the local directory on your Windows PC or Mac;
- `get FILE` copies from the remote directory shown by `pwd` **to** the local directory shown by `lpwd`;
- `put FILE` copies in the opposite direction.

Where did my downloaded file go?

This is one of the most common sources of confusion with SFTP. A file downloaded with `get` is placed in SFTP's *current local directory*. Before downloading, type

```
lpwd
```

SFTP will print the local folder it is using. For example, on Windows it might report something like `C:/Users/student/Downloads`; on a Mac it might report `/Users/student/Downloads`. If you then type `get results.csv`, the file `results.csv` will appear in that folder on your own computer.

A good habit is to choose the local destination *before* transferring files. For example, if you want downloads to go into your normal Downloads folder:

Windows (PowerShell), before starting SFTP:

```
cd $HOME\Downloads
sftp YOUR_USERNAME@130.88.55.57
```

If you are using the older Windows **Command Prompt** rather than PowerShell, the equivalent first command is `cd %USERPROFILE%\Downloads`.

Then, inside SFTP:

```
lpwd
get results.csv
exit
```

Back in PowerShell, open the folder in File Explorer with:

```
explorer .
```

macOS (Terminal), before starting SFTP:

```
cd ~/Downloads
sftp YOUR_USERNAME@130.88.55.57
```

Then, inside SFTP:

```
lpwd
get results.csv
exit
```

Back in Terminal, open the folder in Finder with:

```
open .
```

You can also change the local destination while SFTP is running with `lcd DIR`. After doing so, type `lpwd` again to confirm where files will be saved. If you cannot find a transferred file, do not transfer it repeatedly: reconnect, use `lpwd`, and check that folder first.

9 Getting practical code with Git

Git is a version-control system; GitHub is a service that hosts Git repositories. You do not need to learn advanced Git for the first practical. The main command is `git clone`, which creates a local copy of a repository on the server.

After logging into the modelling server with SSH:

```
cd ~
git clone https://github.com/EnvModelling/approximate-methods-practical
cd approximate-methods-practical
ls
```

If you return to a repository later and have been told that the teaching material has changed, enter the repository and use `git pull` before starting work. Do not use `git pull` blindly if you have made substantial changes of your own; Git may ask you to resolve them.

10 A first-session checklist

Before you begin the scientific part of the first practical, make sure you can do all of the following without assistance:

1. find the current server address and course-specific credentials on Canvas;

2. open PowerShell/Command Prompt/Windows Terminal or macOS Terminal;
3. connect with `ssh YOUR_USERNAME@SERVER_ADDRESS`;
4. recognise that the prompt has changed from your local computer to the remote server;
5. run `hostname`, `pwd` and `ls`;
6. create a directory, enter it and return to your home directory;
7. create and save a short file with `nano -l`;
8. open a *second local terminal* and copy that file back with `scp` or `sftp`;
9. log out of SSH with `exit`.

If you can complete that sequence, you have the core computing skills required for the rest of either module.

11 Troubleshooting

Most connection problems are simple. Read the message in the terminal before trying random fixes.

What you see	What to check
`ssh is not recognised`	On personal Windows computers, check that OpenSSH Client is enabled. On managed computers, ask a GTA rather than changing protected settings.
`Could not resolve hostname`	Usually a typo in the server address. On campus it should be `130.88.55.57`; do not include spaces.
`Connection timed out`	Check your network connection and that you are using `130.88.55.57` on campus. If several students see the same error, ask a GTA because the server may be unavailable.
`Permission denied`	Recheck the course username and password. They are not your central University credentials. Password entry is invisible.
`REMOTE HOST IDENTIFICATION HAS CHANGED`	Stop and ask a GTA or lecturer before bypassing the warning. It can occur after a legitimate server rebuild, but SSH displays it because an unexpected host-key change can also be a security problem.
`No such file or directory`	Run `pwd` and `ls`; you are probably in the wrong directory or have a spelling/capitalisation mismatch.
A command appears to hang	Give it a moment; if you are sure it should not be running, press Ctrl+C.

When asking for help, it is much more useful to show the **exact command and exact error message** than to say only that "the server does not work". Never include your password in a screenshot.

12 Docker: optional way to work locally from home

The normal course workflow is SSH to the modelling server. **Docker is optional.** It is useful if you want a reproducible Linux-based environment on your own computer, particularly when working away from campus or when you want to repeat a practical without relying on the course server.

Docker Desktop is available for Windows and macOS. On current Windows installations it normally uses a Linux environment through WSL 2; on macOS choose the installer appropriate to your Mac. Because Docker Desktop changes frequently, use the current Docker Desktop documentation and any course-specific notes on Canvas rather than relying on old screenshots.

Once Docker Desktop is installed and running, open a local terminal and check:

```
docker --version
```

The course container support is maintained in:

```
git clone https://github.com/UoM-maul1609/projects-and-teaching
cd projects-and-teaching
```

The current repository contains separate launcher scripts for Windows and macOS. The modelling image can be pulled with:

```
docker pull ghcr.io/uom-maul1609/projects-and-teaching-all
```

Then use the launcher for your operating system from the repository root:

```
# Windows PowerShell
.\scripts\docker-run-win.bat

# macOS Terminal
./scripts/docker-run-mac.sh
```

If Canvas gives a different image name or launcher instruction, **Canvas takes precedence**: the container is teaching infrastructure and may be updated during the year.

13 What these tools are for

The command line is only the machinery. The scientific goal is to understand what a model represents and why one model is appropriate for one environmental problem but not another. Across the two modules you will encounter:

- **ODE models**, where the state of a system evolves in time;
- **empirical or semi-empirical models**, which combine theory with parameters constrained by observations;
- **reduced-dimensional models**, which simplify geometry to isolate important processes;
- **fluid-dynamical models**, which solve equations describing motion and transport in fluids such as the atmosphere and ocean.

A question worth asking every time you run a model is: *what has the model retained, what has it neglected, and are those choices appropriate for the problem?*